

Bharat Badgujar

Linux module 2

Assignment 6

1.

Install Apache web server and PHP on your Linux machine, then create an index.php file that displays 'Welcome to My Foodie Blog' when accessed via your browser.

Ans : **Install Apache & PHP**

sudo apt update

sudo apt install apache2 php libapache2-mod-php -y

Create index.php:

sudo nano /var/www/html/index.php

```
<?php
```

```
echo "Welcome to My Foodie Blog";
```

```
?>
```

Open in browser:

http://localhost/index.php

2.

Configure your Apache server to serve a custom 404 error page that displays a Zomato-style message: 'Oops! This page is as empty as your cart.'

Ans : Create file:

sudo nano /var/www/html/404.html

```
<h1>Oops! This page is as empty as your cart.</h1>
```

Edit Apache config:

sudo nano /etc/apache2/apache2.conf

Add:

```
ErrorDocument 404 /404.html
```

Restart Apache:

```
sudo systemctl restart apache2
```

3.

Set up a simple FTP server (vsftpd or any other) on your Linux system and upload a sample image file to your web server's root directory using an FTP client.

Ans : **Install FTP Server**

```
sudo apt install vsftpd -y
```

```
sudo systemctl enable --now vsftpd
```

Upload an image using an FTP client (e.g., FileZilla) to:

```
/var/www/html/
```

4.

Configure your web server to also run a basic Node.js service on a different port (e.g., 3000) that returns JSON data for a mock 'Now Playing' playlist, similar to Spotify. Test accessing both the PHP web page and the Node.js playlist endpoint from your browser.

Ans : **Run Node.js Service**

Install Node.js:

```
sudo apt install nodejs npm -y
```

Create server.js:

```
const http = require('http');
```

```
http.createServer((req, res) => {  
  res.writeHead(200, {'Content-Type': 'application/json'});  
  res.end(JSON.stringify({  
    "Now Playing": "My Playlist"  
  }));  
}).listen(3000);
```

Run:

```
node server.js
```

Test:

```
http://localhost/index.php
```

```
http://localhost:3000
```

5.

Edit your Apache configuration to restrict access to a specific directory (e.g., /var/www/html/secret) so that only users from localhost (127.0.0.1) can access it.

Hint: Use the 'Require' directive in your Apache config or .htaccess file.

Ans : **Restrict Access to /secret**

Create directory:

```
sudo mkdir /var/www/html/secret
```

Edit Apache config:

```
sudo nano /etc/apache2/apache2.conf
```

Add:

```
<Directory "/var/www/html/secret">
```

```
    Require ip 127.0.0.1
```

```
</Directory>
```

Restart Apache:

```
sudo systemctl restart apache2
```